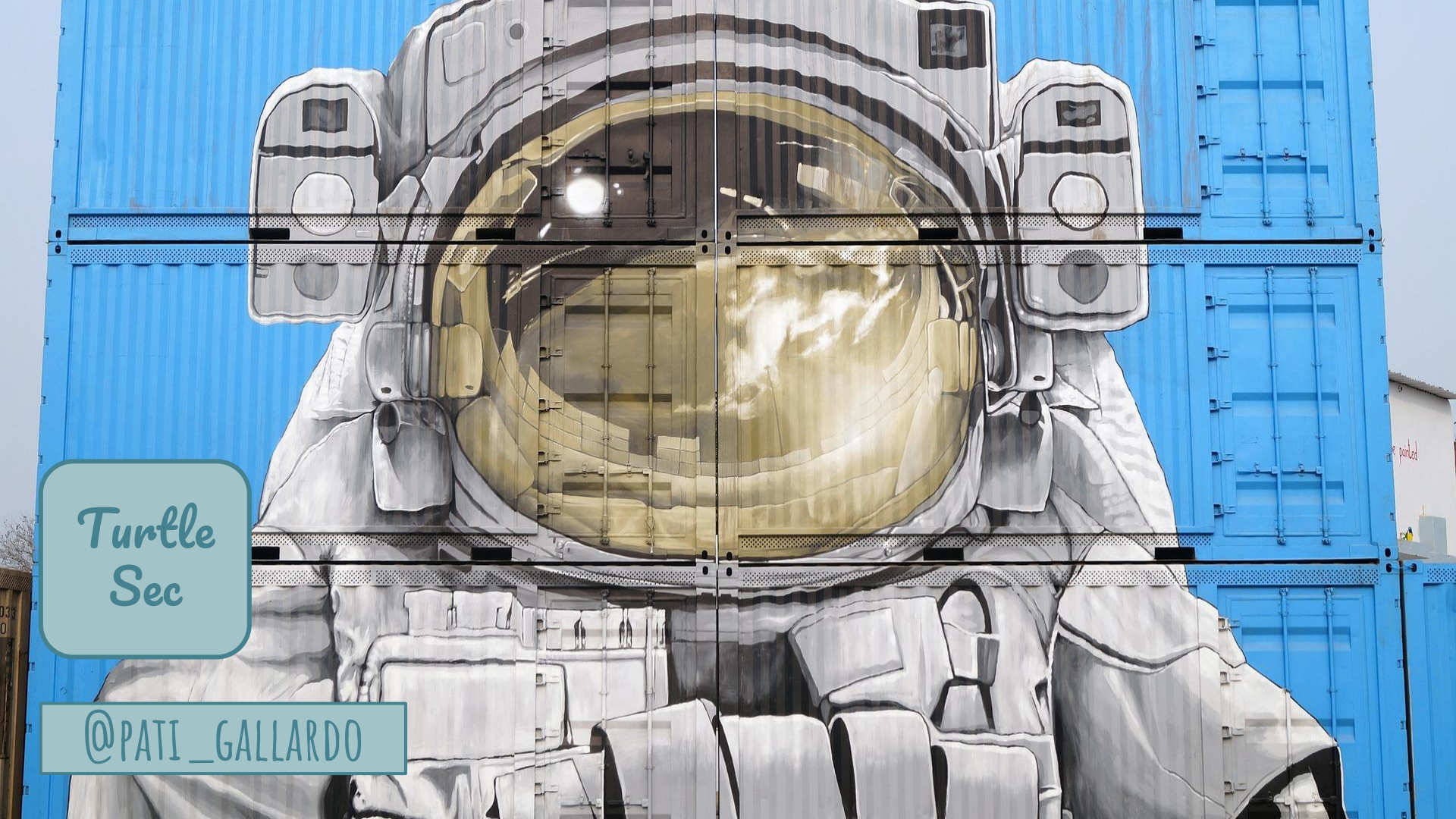


A large mural of a turtle wearing a white space suit with a large, clear, yellowish-gold visor. The turtle is positioned in front of a wall of blue shipping containers. The visor reflects the surrounding environment, including the containers and a bright light source. The turtle's suit has various straps and pockets. The background consists of several blue shipping containers stacked vertically. On the right side, a small portion of a white building with red lettering is visible.

*Turtle
Sec*

@PATI_GALLARDO

@PATI_GALLARDO

LINUX SECURITY

and the Chromium Sandbox

PATRICIA AAS, *TurtleSec*
BlackHoodie 2018

Turtle
Sec

PATRICIA AAS - CONSULTANT

@PATI_GALLARDO

C++ Programmer, Application Security

Currently : *TurtleSec*

Previously : Vivaldi, Cisco Systems, Knowit, Opera Software

Master in Computer Science - main language Java

Pronouns: she/her

The logo for TurtleSec, featuring the words "Turtle" and "Sec" stacked vertically in a teal, handwritten-style font, enclosed within a teal rounded square border.

*Turtle
Sec*

Remote Code Execution
is what browsers do.

SANDBOXING

@PATI_GALLARDO

- System External Threat : Protect the system from the vulnerabilities in the browser
- System Internal Threat : Protect the browser from malware on the system





OUTLINE

- Process Architecture
- Linux Security APIs
- The Initial Sandbox
- Shrinking the Sandbox

@PATI_GALLARDO

@PATI_GALLARDO

Process Architecture



THE EXECUTABLE FILES

- **vivaldi** : The bash script wrapper, launches vivaldi-bin. Sets up environment.
- **vivaldi-bin** : The browser binary
- **vivaldi-sandbox** : A setuid binary, not in much use today



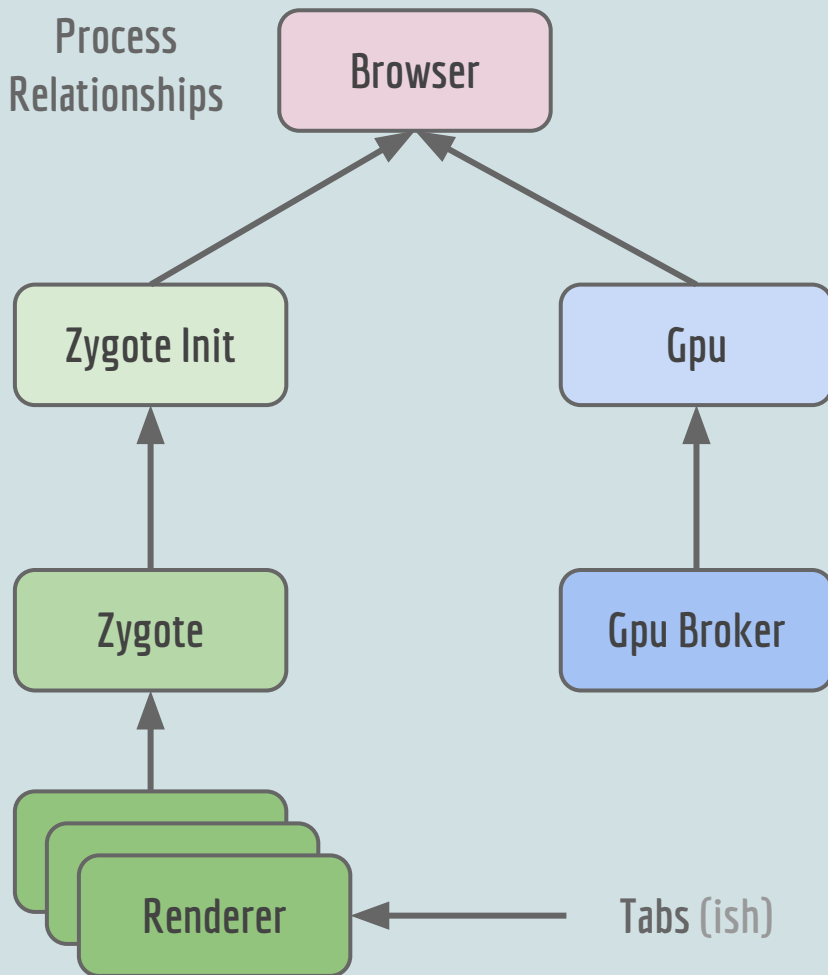


PROCESS ARCHITECTURE

@PATI_GALLARDO

9

Process
Relationships



THE RUNNING PROCESSES

1. Browser :
vivaldi-bin
2. Zygote (Parent) :
vivaldi-bin --type=zygote
3. Zygote (Child) :
vivaldi-bin --type=zygote
4. Renderer (MANY) :
vivaldi-bin --type=renderer
5. GPU (Parent) :
vivaldi-bin --type=gpu-process
6. GPU Broker (Child) :
vivaldi-bin --type=gpu-broker

@PATI_GALLARDO





OUTLINE

- Process Architecture
- **Linux Security APIs**
- The Initial Sandbox
- Shrinking the Sandbox

@PATI_GALLARDO

Linux Security APIs

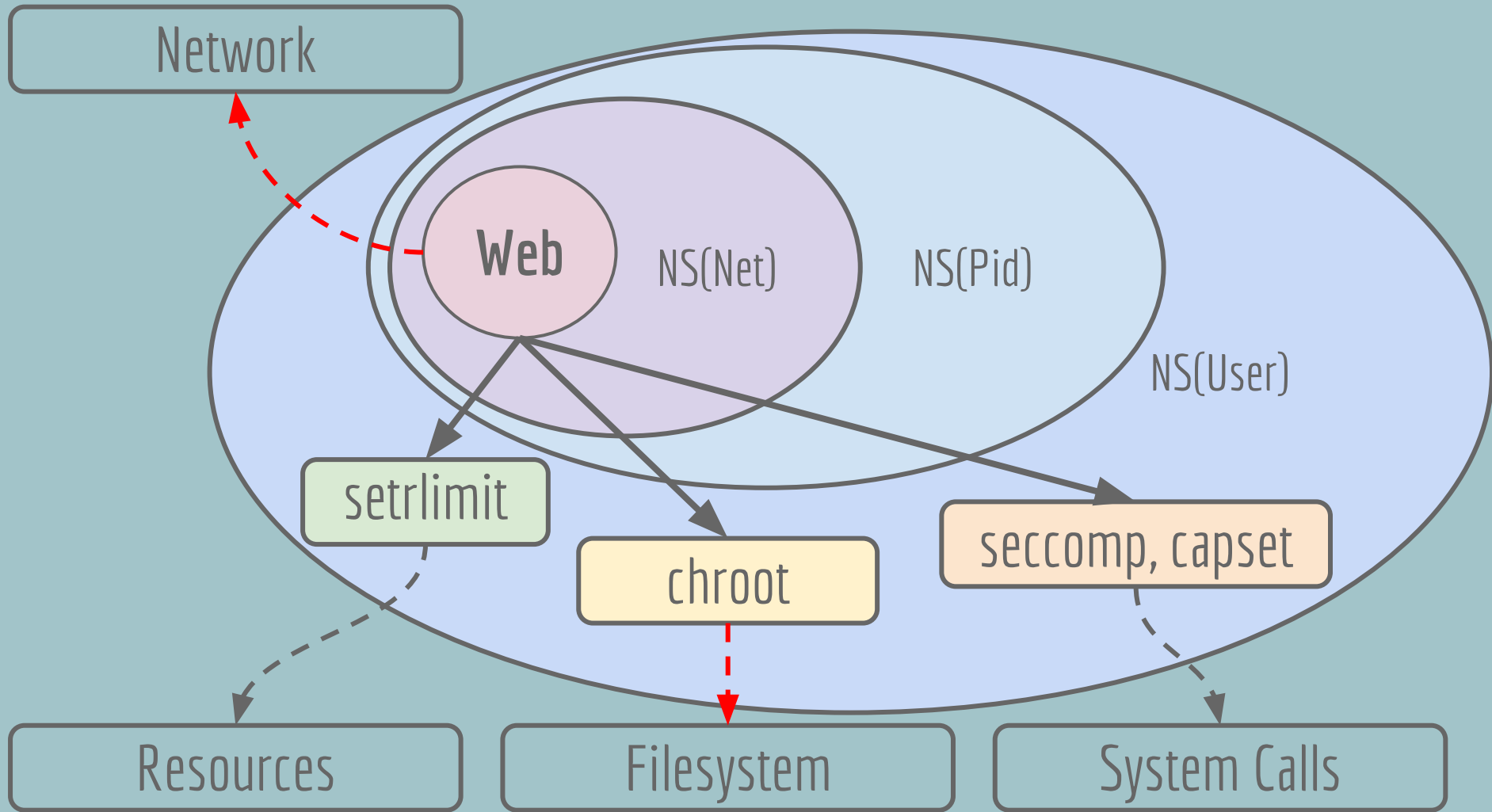
CHROME://SANDBOX

Sandbox Status

SUID Sandbox	No
Namespace Sandbox	Yes
PID namespaces	Yes
Network namespaces	Yes
Seccomp-BPF sandbox	Yes
Seccomp-BPF sandbox supports TSYNC	Yes
Yama LSM Enforcing	Yes

You are adequately sandboxed.

@PATI GALLARDO





OUTLINE

- Process Architecture
- Linux Security APIs
- **The Initial Sandbox**
- Shrinking the Sandbox

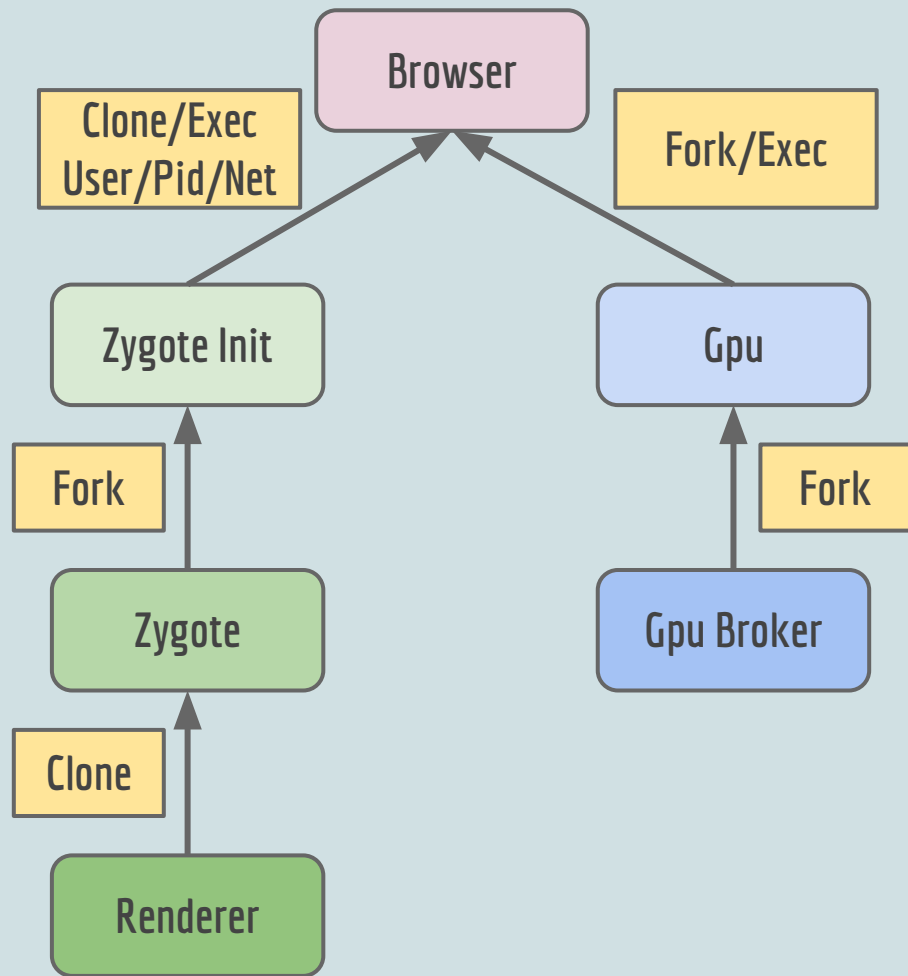
@PATI_GALLARDO

The Initial Sandbox

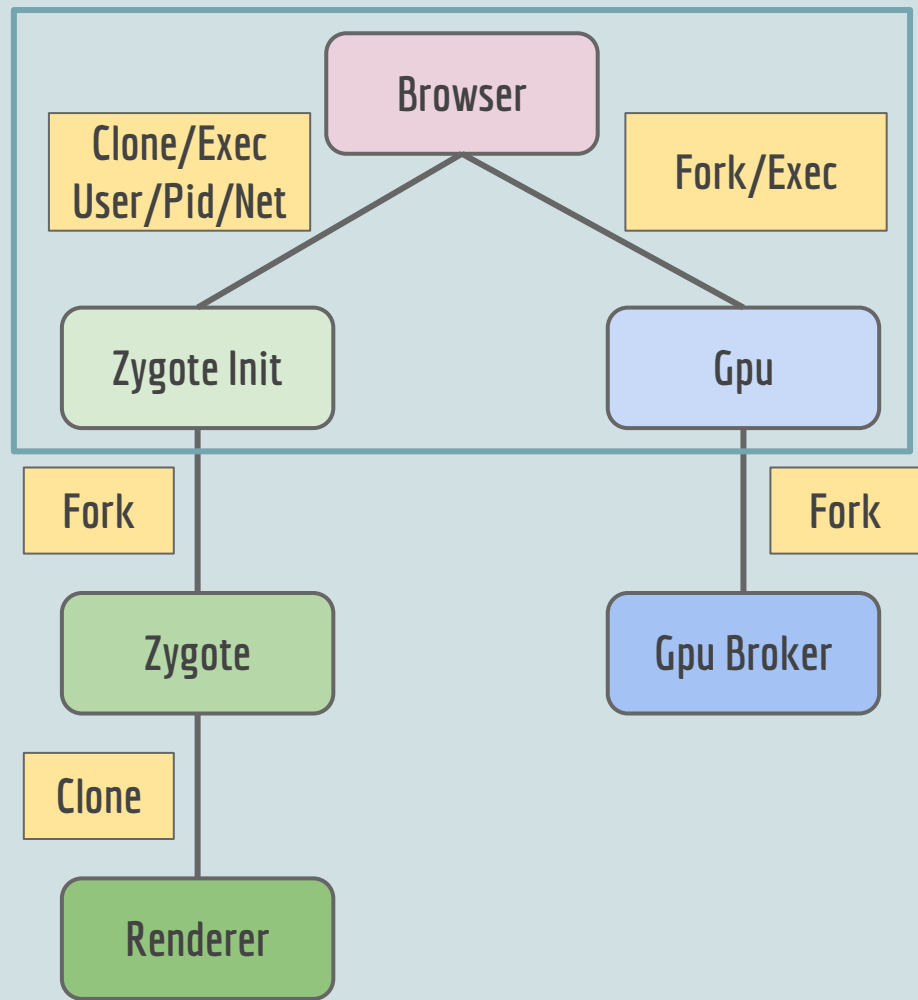
ONE BINARY

@PATI_GALLARDO

17



INITIAL SANDBOX



@PATI_GALLARDO

FORK / EXEC

- A typical start of a new process on Linux is a “**fork/exec**”
- “**Forking**” is creating a new process
- “**Exec**” is executing a binary in a process
- “**Clone**” is a type of fork which can restrict a process at creation

@PATI_GALLARDO

SANDBOXING OPPORTUNITIES: FORK

1. BEFORE FORK

2. AFTER FORK

SANDBOXING OPPORTUNITIES: "FORK"/EXEC

1. BEFORE CLONE/FORK

2. (AT CLONE)

3. BEFORE EXEC

4. AT STARTUP (INPUT : ARGV, ENV)

Namespaces

@PATI_GALLARDO

22



NAMESPACES

(ZYGOTES/RENDERERS)

Limits what a process can see

Api : clone/unshare





ZYGOTE + RENDERER

@PATI_GALLARDO

24

NAMESPACES IN USE

CLONE_NEWUSER

Inside a USER NS we can create a PID NS.

CLONE_NEWPID

Same PID number can represent different processes in different PID namespaces. One init (PID 1) process per PID NS

CLONE_NEWNET

Isolate a process from network



ZYGOTE + RENDERER

@PATI_GALLARDO

25

AT CLONE : CREATE NAMESPACES

Clone flags define the process* created and will create namespaces (NS) for it

1. Test which NS are available
2. Fail if not sufficient
3. Construct the biggest supported and applicable set

Emulates fork with longjmp

* Also used to create threads

@PATI_GALLARDO

```
int flags = CLONE_NEWUSER | CLONE_NEWPID | CLONE_NEWNET;  
jmp_buf env;  
if (setjmp(env) == 0) {  
    return CloneAndLongjmpInChild(flags, ptid, ctid, &env);  
}
```

[chromium/base/process/launch_posix.cc](#)

@PATI_GALLARDO

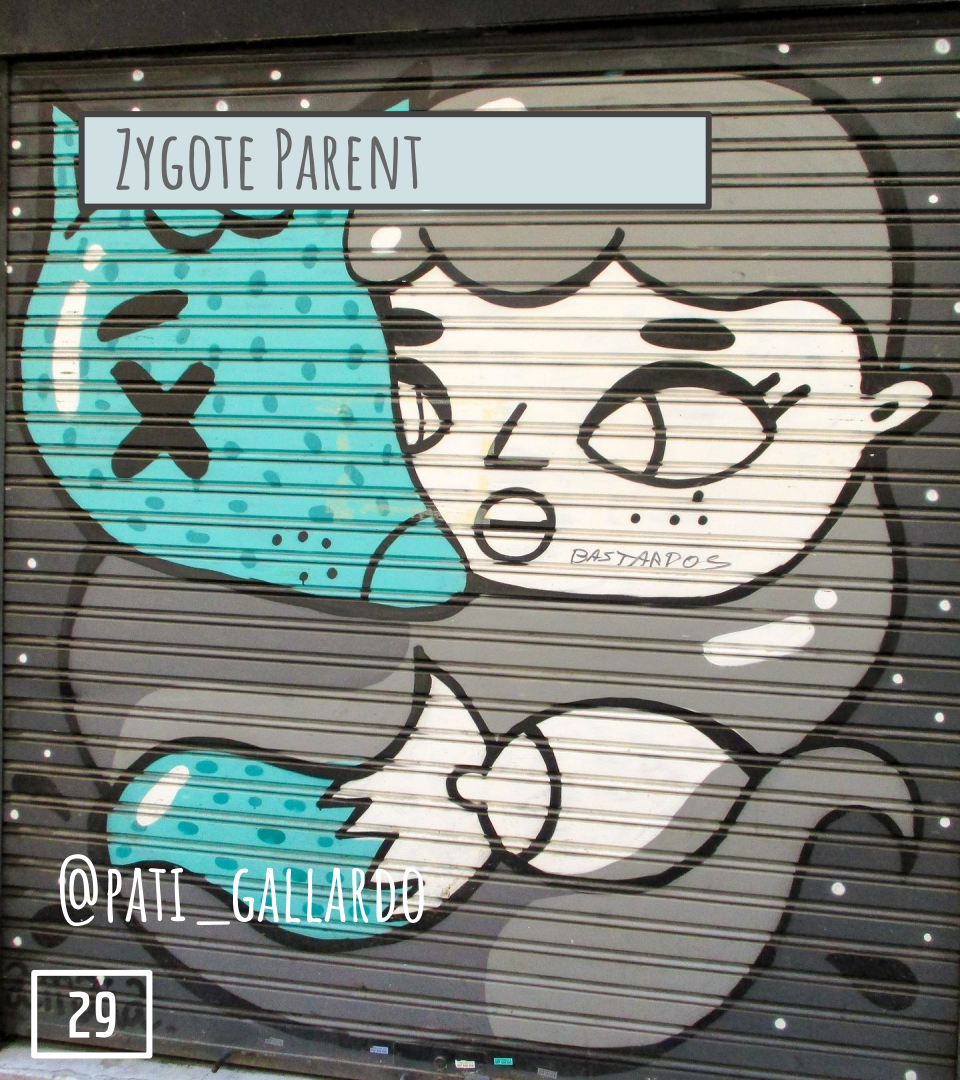
```
pid_t CloneAndLongjmpInChild(unsigned long flags,  
                             pid_t* ptid,  
                             pid_t* ctid,  
                             jmp_buf* env) {  
    char stack_buf[PTHREAD_STACK_MIN];  
    void* stack = stack_buf + sizeof(stack_buf);  
    return clone(&CloneHelper,  
                stack, flags, env, ptid, nullptr, ctid);  
}
```

chromium/base/process/launch_posix.cc

@PATI_GALLARDO

```
int CloneHelper(void* arg) {  
    jmp_buf* env_ptr = reinterpret_cast<jmp_buf*>(arg);  
    longjmp(*env_ptr, 1);  
    // Should not be reached  
    assert(false);  
    return 1;  
}
```

[chromium/base/process/launch_posix.cc](#)



ZYGOTE PARENT

UNSHARE- USER NAMESPACE

unshare(CLONE_NEWUSER)

Alternative to clone(CLONE_NEWUSER), will **not create new process**, but move caller.

Credentials::CanCreateProcessInNewUserNS

@PATI_GALLARDO

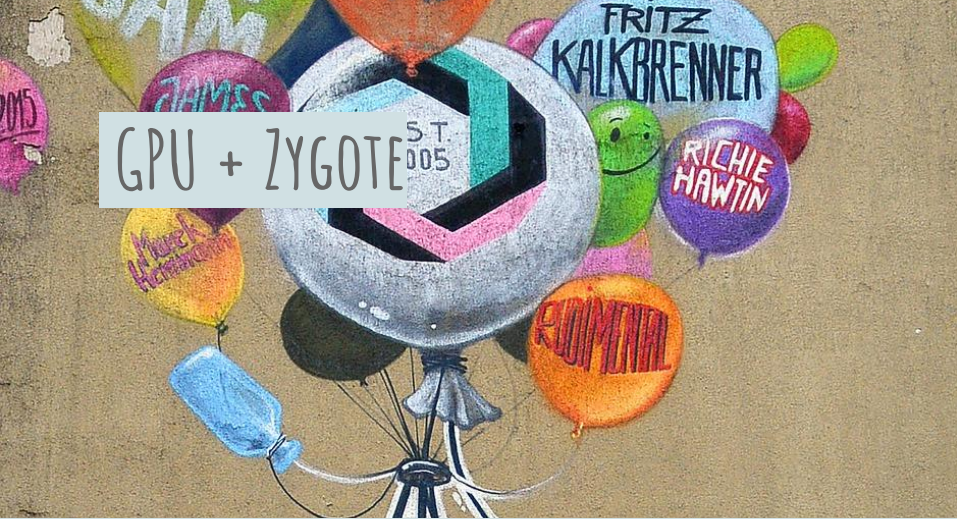
SANDBOXING OPPORTUNITIES: "FORK"/EXEC

1. BEFORE CLONE/FORK

2. AT CLONE

3. BEFORE EXEC

4. AT STARTUP (INPUT : ARGV, ENV)



GPU + ZYGOTE

BEFORE EXEC : LAUNCH OPTIONS



@PATI_GALLARDO

31

PREPARE FOR EXEC

1. Fix the environment
2. Fix file descriptors
3. Fix signal handling
4. Set up process group
5. Maximize resource limits
6. Set `PR_SET_NO_NEW_PRIVS`
7. Change current dir
8. Select executable path
9. Setup command-line



OUTLINE

- Process Architecture
- Linux Security APIs
- The Initial Sandbox
- **Shrinking the Sandbox**

@PATI_GALLARDO

Shrinking the Sandbox

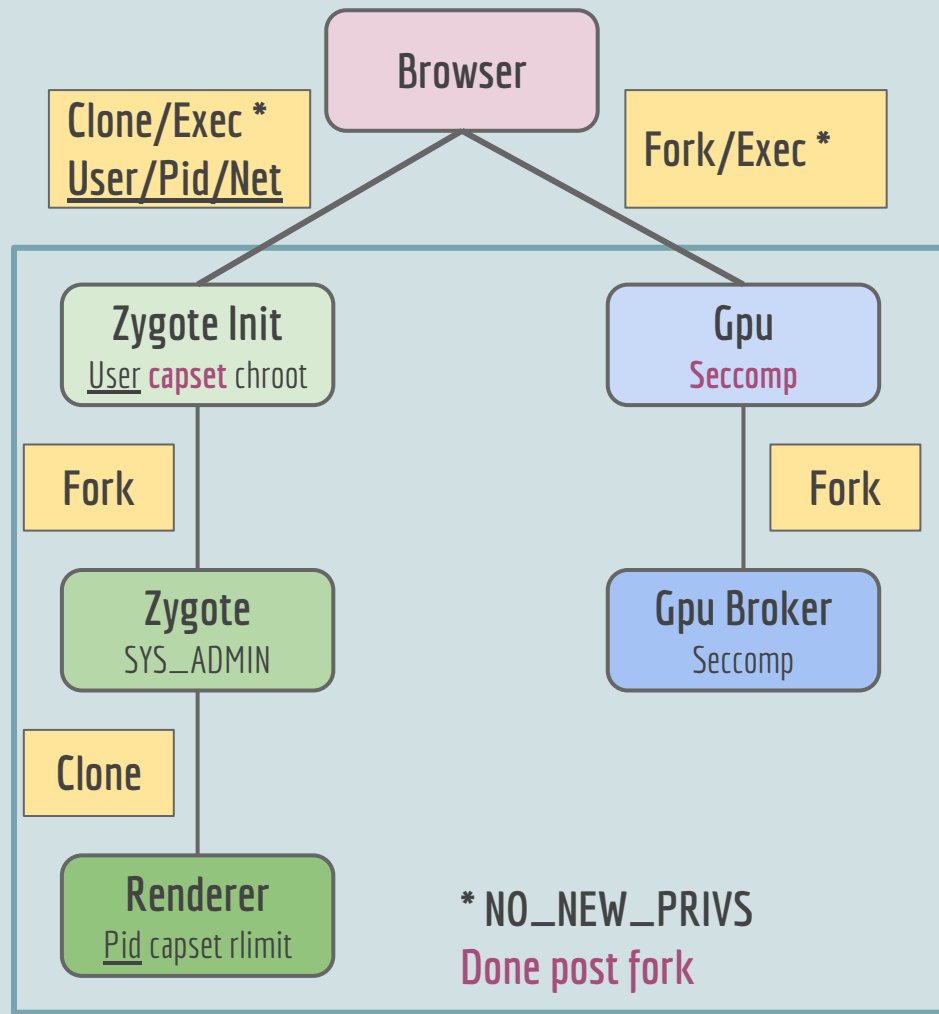


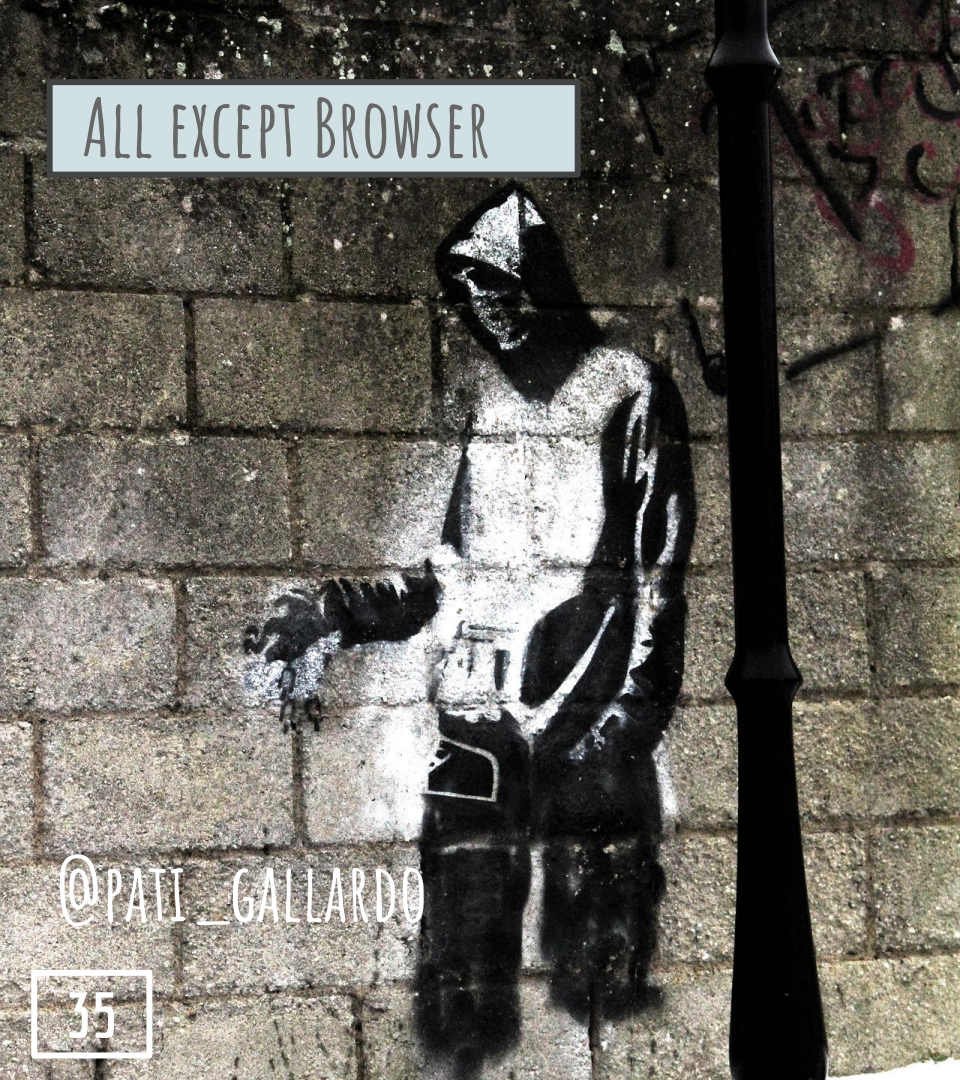
SHRINKING THE SANDBOX



@PATI_GALLARDO

34





ALL EXCEPT BROWSER

PR_SET_NO_NEW_PRIVS

`prctl(PR_SET_NO_NEW_PRIVS)`

Preserved across fork, clone and execve

“If `no_new_privs` is set, then **operations that grant new privileges (i.e. execve) will either fail or not grant them.** This affects suid/sgid, file capabilities, and LSMs.”

`/usr/include/linux/prctl.h`

@PATI_GALLARDO

Seccomp

@PATI_GALLARDO

36

@PATI_GALLARDO



SECCOMP

(RENDERERS/GPU/BROKER)

Limits which syscalls a process
can call and how these calls are
handled

Api : seccomp



@PATI_GALLARDO



RENDERER + GPU + BROKER

@PATI_GALLARDO

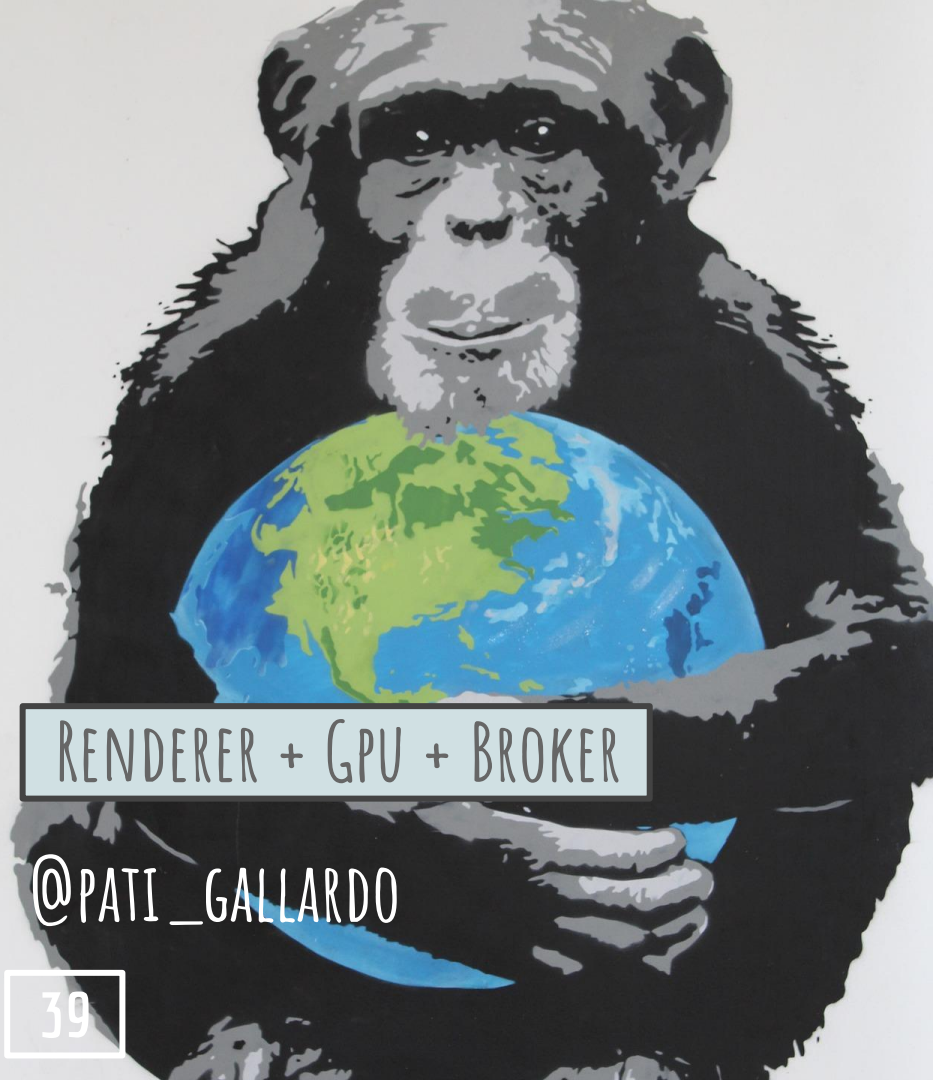
38

SECCOMP BPF PROGRAM

Program written in an assembly-like language to filter system-calls.

Runs in a simple VM in kernel space. All syscalls will be filtered by this program

TSYNC : Once a Seccomp Program is installed it applies to all threads in a process



RENDERER + GPU + BROKER

@PATI_GALLARDO

39

SECCOMP : BPF POLICIES

BPF Program defined in a Policy

Fundamentally a whitelist, allows a set of syscalls and has custom handling of others.

An extended Policy is generally more permissive

1. BaselinePolicy

1.1 GpuProcessPolicy

1.1.1 GpuBrokerProcessPolicy

1.2 RendererProcessPolicy

@PATI_GALLARDO

```
void StartSandboxWithPolicy(sandbox::bpf_dsl::Policy* policy)
{
    SandboxBPF sandbox(policy);
    assert(sandbox.StartSandbox());
}

bool SandboxBPF::StartSandbox() {
    InstallFilter();
    return true;
}
```

chromium/services/service_manager/sandbox/linux/sandbox_seccomp_bpf_linux.cc
chromium/sandbox/linux/seccomp-bpf/sandbox_bpf.cc

@PATI_GALLARDO

```
void SandboxBPF::InstallFilter() {  
    CodeGen::Program program = AssembleFilter();  
    struct sock_filter bpf[program.size()];  
    const struct sock_fprog prog =  
        { static_cast<unsigned short>(program.size()), bpf };  
    memcpy(bpf, &program[0], sizeof(bpf));  
    assert(prctl(PR_SET_NO_NEW_PRIVS, 1, 0, 0, 0) == 0);  
    assert(seccomp(SECCOMP_SET_MODE_FILTER,  
                  SECCOMP_FILTER_FLAG_TSYNC, &prog) == 0);  
}
```

[chromium/sandbox/linux/seccomp-bpf/sandbox_bpf.cc](https://chromium.org/sandbox/linux/seccomp-bpf/sandbox_bpf.cc)

Chroot

@PATI_GALLARDO

42

@PATI_GALLARDO



CHROOT

(ZYGOTES/RENDERERS)

Limits what a process can see of
the filesystem

Api : chroot





ZYGOTES + RENDERER

@PATI_GALLARDO

CHROOT : DROP ACCESS TO FS

- `clone(CLONE_FS)` a child process
- Child `chroot("/proc/self/fdinfo/")`
- Child immediately does a `chdir("/")`
- Child does `_exit(kExitSuccess)`

You can see this by looking at

`ls -l /proc/<pid>/root`

Of the Zygote or any ancestor

Credentials::DropFileSystemAccess

@PATI_GALLARDO

```
bool ChrootToSafeEmptyDir() {  
    pid_t pid = -1;  
    char stack_buf[PTHREAD_STACK_MIN];  
    void* stack = stack_buf + sizeof(stack_buf);  
    int clone_flags = CLONE_FS | LINUX_SIGCHLD;  
    pid = clone(ChrootToSelfFdinfo, stack, clone_flags, nullptr);  
    int status = -1;  
    assert(HANDLE_EINTR(waitpid(pid, &status, 0)) == pid);  
    return WIFEXITED(status) && WEXITSTATUS(status) == kExitSuccess;  
}
```

chromium/sandbox/linux/services/credentials.cc

@PATI_GALLARDO

```
int ChrootToSelfFdinfo(void*) {  
    assert(chroot("/proc/self/fdinfo/") == 0);  
    assert(chdir("/") == 0);  
    _exit(kExitSuccess);  
}
```

chromium/sandbox/linux/services/credentials.cc

Capabilities



@PATI_GALLARDO

47

@PATI_GALLARDO

CAPABILITIES

(ZYGOTE/RENDERERS)

Limits what a process is
privileged enough to do

Api : capset



@PATI_GALLARDO



ZYGOTES + RENDERERS

@PATI_GALLARDO

49

DROP CAPABILITIES

Uses `capset()` to drop all or some capabilities

“Linux divides the privileges traditionally associated with superuser into distinct units, known as capabilities, which can be independently enabled and disabled.”

Man page for capabilities

Credentials::DropAllCapabilities

Resource Limits

@PATI_GALLARDO

50



RESOURCE LIMITS

(RENDERERS)

Limits the access this process has to platform resources

Api : setrlimit





RENDERER

@PATI_GALLARDO

52

RESOURCE LIMITS : SETRLIMIT

Limits using setrlimit:

1. **RLIMIT_AS** : Maximum size of the process' virtual memory (address space) in bytes
2. **RLIMIT_DATA** : Maximum size of the process's data segment

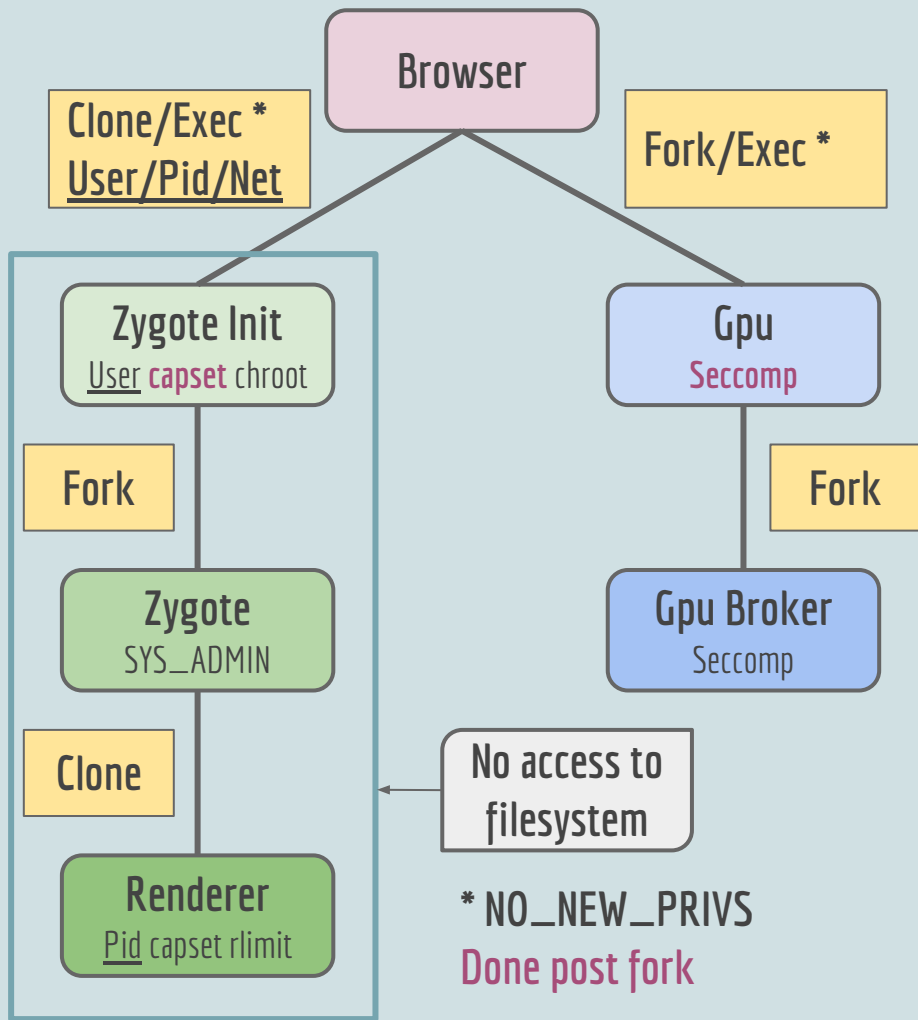
LinuxSandbox::LimitAddressSpace



TRUST IS RELATIVE

@PATI_GALLARDO

53



CHROME://SANDBOX



@PATI_GALLARDO

54

Sandbox Status

SUID Sandbox	No
Namespace Sandbox	Yes
PID namespaces	Yes
Network namespaces	Yes
Seccomp-BPF sandbox	Yes
Seccomp-BPF sandbox supports TSYNC	Yes
Yama LSM Enforcing	Yes

You are adequately sandboxed.

@PATI_GALLARDO

SOURCES

Chromium/Kernel source + Linux Man Pages + lwn.net

MICHAEL KERRISK

Book: *The Linux Programming Interface*

Course: *Linux Security and Isolation APIs*

ALL ERRORS ARE MY OWN

@PATI_GALLARDO

Patricia Aas, Consultant
TurtleSec

C++ and Application Security

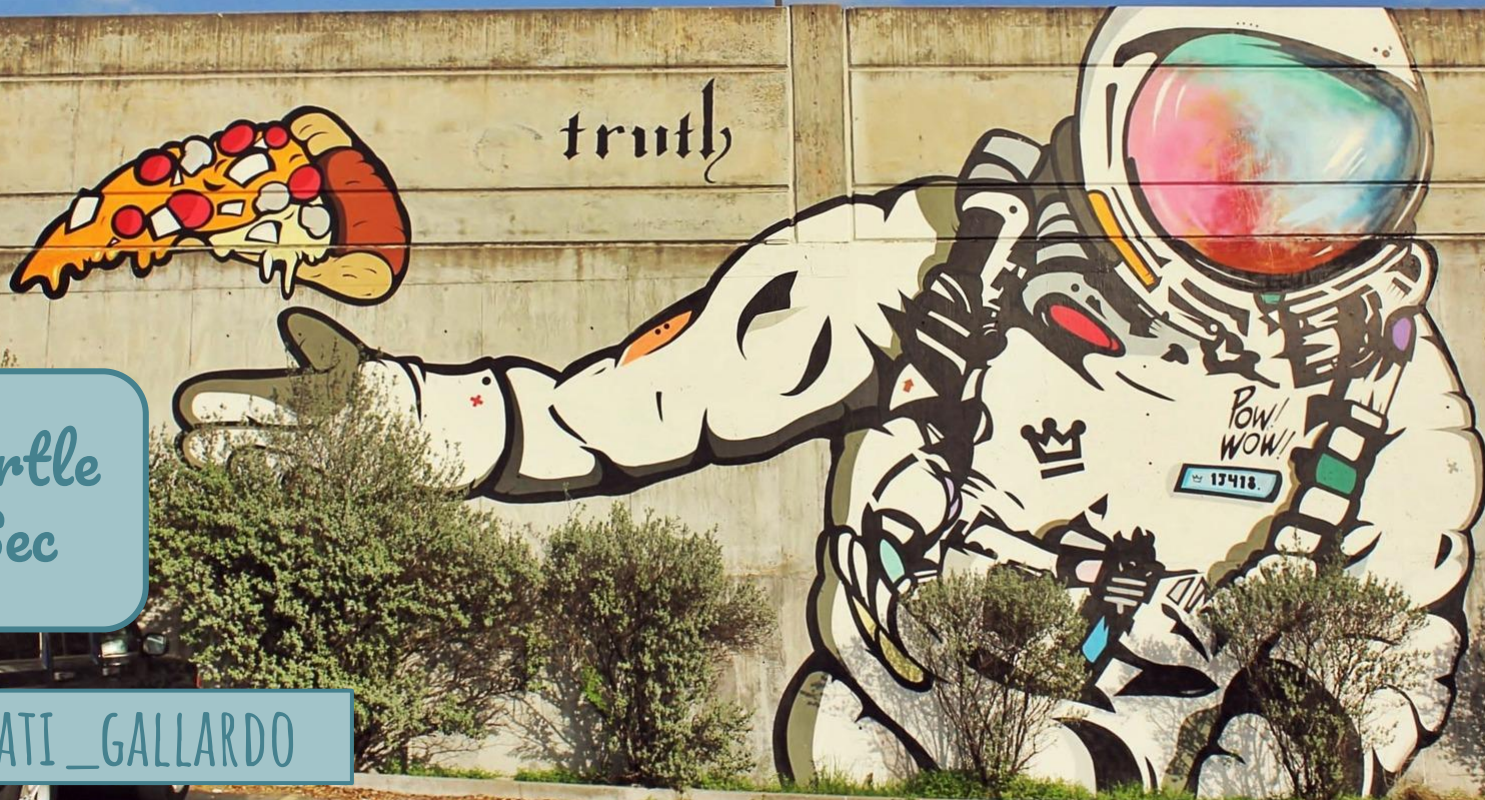
*Turtle
Sec*

@PATI_GALLARDO

truth

Turtle
Sec

@PATI_GALLARDO



@PATI_GALLARDO

APPENDIX / SOME NOTES

Other APIs
in use

Not Used Much in Chromium, but...

YAMA LSM

Limits the access that other process have to this process - especially ptracing

Status is checked by reading :
`/proc/sys/kernel/yama/ptrace_scope`



SETUID/SETGID

(LEGACY)

Increases what a process is privileged enough to do, by using the privilege of the executables owner

Api : `set*uid / set*gid`



CGROUPS

(CHROMEOS)

Limits the resources available to a process. Used in ChromeOS - not covered here.

`/proc/⟨PID⟩/cgroup`
`/proc/cgroups`



PROCESS GROUPS

(CHROMEDRIVER)

Can be used to treat a group of processes as one.

Used with the 'detach' property of Chromedriver

Api : `setpgid`



Challenges



Debugging

@PATI_GALLARDO



118

Crash Reporting

@PATI GALLARDO

Photos from pixabay.com

Patricia Aas, *TurtleSec*
@pati_gallardo

*Turtle
Sec*

@PATI_GALLARDO

Turtle
Sec

@PATI_GALLARDO

